INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

EcoGuard: IoT-Based Environmental Monitoring and Alert System

FINAL YEAR

PROJECT PHASE - 1 REPORT

Submitted by

Jagan M (23619116)
Mohammed Abrar S (23619121)
Parthasarathy R (23619127)

in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY CSE –

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES

(VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "Confidence-Gated Explainable AI for Reliable Churn Prediction and Automated Retention Decisions", is the Bonafide work of **Jagan M (23619116), Mohamed Abrar (23619121), Parthasarathy R (23619127)** carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD

Dr. K. Ulagapriya M.E., Ph.D.

Associate Professor & Head

Department of AI&ML

School of Engineering

VISTAS

Chennai – 600 117

SIGNATURE OF THE SUPERVISOR

Ms. S.J. Kavitha ME, Ph.D.

Mr. S. Akash M.Tech

Assistant Professor

Department of AI&ML

School of Engineering

VISTAS

Chennai – 600 117

Submitted for the Project Work and Viva - Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this project a successful one. The infrastructural support and laboratory facilities provided by our institution have been a motivating factor in completing this project work. With great pleasure, I take this opportunity to thank our Founder and Chairman, Shri. Dr. Ishari K. Ganesh, for providing the necessary facilities and support. I am grateful to our Head of the Department, Dr. K. Ulagapriya, for her continuous support, valuable guidance, and encouragement throughout the project. I extend my sincere thanks to our Project Guide, Ms. S. J. Kavitha and Mr. S. Akash for patient guidance, valuable suggestions, and continuous assistance throughout the project. I am also thankful to our Project Coordinator, Ms. S. J. Kavitha, for her support and guidance. Her encouragement and support have been instrumental in the successful completion of this work. I would also like to express my heartfelt gratitude to all the faculty members, the librarian, and the non-teaching staff of the Computer Centre for their help and support throughout the completion of this project. Last but not least, I wish to thank my parents and friends for their constant support, encouragement, and motivation, which inspired us to overcome challenges and complete this project successfully.

JAGAN M **23619116**

MOHAMMED ABRAR S **23619121**

PARTHASARATHY R **23619127**

TABLE OF CONTENTS

CHAPTER NO. TITLE PAGE NO.

ABSTRACT.....i
LIST OF FIGURES.....ii
LIST OF TABLES.....iii
LIST OF ABBREVIATIONS.....iv

1 INTRODUCTION.....**3**
1.1 Overview of the Project.....3
1.2 Problem Statement.....4
1.3 Objectives of the Project.....5
1.4 Scope of the Project.....6
1.5 Organization of the Report.....6

2 LITERATURE REVIEW.....**7**
2.1 Literature Overview.....7
2.2 Literature Survey.....8
2.3 Review of Existing IoT Monitoring Systems.....10
2.4 Research Gap.....12
2.5 Summary.....13

3 SYSTEM DESIGN.....**13**
3.1 Existing System.....13
3.2 Proposed System.....14
3.3 Dataset Description.....15
3.4 System Architecture.....17
3.5 Hardware Connection Setup.....19
3.6 Hardware Requirements.....20
3.7 Software Requirements.....22

4 IMPLEMENTATION.....**24**
4.1 Methodology.....24
4.2 ESP32 Configuration.....25
4.3 DHT22 Temperature and Humidity Monitoring.....26
4.4 MQ-135 Gas Sensor Integration.....27
4.5 NEO-6M GPS Integration.....29
4.6 Buzzer-Based Alert Mechanism.....31
4.7 Serial Communication with the Web Application.....33
4.8 Real-Time Dashboard Implementation.....35
4.9 Historical Data and Sensor-Wise Monitoring.....36
4.10 Report Generation and Data Export.....37

5 RESULTS AND DISCUSSION.....39
5.1 Testing Methodology.....39
5.2 Individual Sensor Testing.....40
5.3 Integrated Hardware Testing.....40
5.4 Dashboard Functionality Testing.....41
5.5 Buzzer Alert Testing.....41
5.6 GPS Fix and Location Testing.....42
5.7 Results and Discussion.....43

6 CONCLUSION AND FUTURE WORK.....44
6.1 Conclusion.....44
6.2 Future Work.....45

7 SOURCE CODE.....47

8 REFERENCES.....56

ABSTRACT

Air pollution continues to be one of the most serious environmental and public health problems in the world, impacting millions of people. Poor air quality increases the risk for vulnerable populations, including children, the elderly, pregnant women, and persons with respiratory or cardiovascular diseases. Current air quality monitoring systems usually report levels in terms of the Air Quality Index (AQI), which is difficult for the general public to interpret and does not provide personalized health advice. The project is an AI-Powered Personalized Environmental Health Advisory System Using IoT, which will continuously monitor environmental conditions and provide customized health advisories based on the health profile of the user. The system will be built around an ESP32 microcontroller, equipped with a PMS5003 (PM2.5/PM10) sensor, MQ-135 gas sensor, DHT22 temperature and humidity sensor, and a GPS module for real-time environmental data collection. The collected data is sent to a cloud platform through Wi-Fi, where an AI model analyzes the sensor readings and produces an Environmental Health Risk Score (EHRS). The system produces personalized health recommendations based on the EHRS and the user's chosen health profile. These include the safety of staying at the present location, how long to remain there, and precautions to take. The aim of the system is to generate user-specific health guidance from complex air quality data in an easy-to-understand format, enabling vulnerable individuals to make informed decisions and reduce their health risks from exposure to air pollution.

KEYWORDS -- ESP32 Development Board, PMS5003 PM2.5/PM10 Sensor, MQ-135 Gas Sensor, DHT22 Temperature and Humidity Sensor, GPS Module (NEO-6M).

LIST OF FIGURES

FIGURE NO	TITLE LIST OF FIGURES	PAGE NO
Figure 1	System Architecture of the EcoGuard IoT Environmental Monitoring System	17
Figure 2	EHRS Risk Category	34
Figure 3	Time Vs Environmental Sensor Readings	35
Figure 4	Integrated Hardware Setup	39
Figure 5	EcoGuard Environmental Monitoring Dashboard	40

LIST OF TABLES

Table No.	Title	Page No.
Table 1	Comparison of Environmental Monitoring Techniques	11
Table 3.1	Hardware Pin Connections	19
Table 3.2	Software Requirements	21
Table 3.3	Hardware Requirements	22
Table 4	Functional Test Cases for EcoGuard	34

LIST OF ABBREVIATIONS

Abbreviation	Full Form
IoT	Internet of Things
ESP32	Espressif Systems 32-bit Microcontroller Series
DHT22	Digital Temperature and Humidity Sensor
GPS	Global Positioning System
EHRS	Environmental Health Risk Score
AQI	Air Quality Index
ADC	Analog-to-Digital Converter
GPIO	General-Purpose Input/Output
AIoT	Artificial Intelligence of Things
LSTM	Long Short-Term Memory
PPM	Simple Mail Parts Per MillionProtocol
TX	Transmit

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Introduction

The Internet of Things (IoT) has become an important technology for monitoring environmental conditions through connected sensors and embedded systems. Environmental monitoring helps observe changes in temperature, humidity and air quality that may affect human health and the surrounding environment. Traditional monitoring methods often require manual measurements and may not provide continuous access to environmental data.

The EcoGuard IoT Environmental Monitoring System is designed to monitor environmental conditions using an ESP32 microcontroller and multiple sensors. The DHT22 sensor measures temperature and humidity, while the MQ-135 sensor provides analog readings related to surrounding air conditions. The NEO-6M GPS module provides geographical coordinates when a valid satellite fix is available. A buzzer provides a local warning when the temperature exceeds a predefined threshold of 34°C.

The ESP32 collects sensor readings and transmits them through serial communication to a web-based monitoring application. The dashboard displays current sensor values, GPS information and alerts. It also supports historical data review and data export, helping users observe and analyze collected measurements.

The system combines sensor-based data collection, embedded processing, local alerts and web-based visualization in a single platform. The PMS5003 particulate matter sensor is planned as a future extension to enhance the system's air-quality monitoring capabilities.

Problem Statement

Conventional environmental monitoring systems may have the following limitations:

- Environmental measurements often require manual observation.
- Continuous monitoring may be difficult without automated data collection.
- Sensor readings may be scattered across different devices.
- Users may lack a centralized dashboard for viewing current measurements.
- Environmental changes may go unnoticed without automatic warnings.

- Historical data may be difficult to store and analyze.
- Location information may not be available alongside sensor readings.

Therefore, an IoT-based system is required to collect environmental measurements, display them through a web application, provide threshold-based warnings and associate available GPS coordinates with the collected data.

Objectives of the Project

The major objectives of the EcoGuard project are:

- To develop an IoT-based environmental monitoring system using ESP32.
- To measure temperature and humidity using the DHT22 sensor.
- To monitor air-quality-related changes using the MQ-135 sensor.
- To integrate the NEO-6M GPS module for location tracking.
- To maintain historical readings and support CSV data export.
- To provide a modular design for integrating additional sensors in the future.

1.2 SCOPE AND OBJECTIVES

Purpose of the Study

The purpose of this project is to develop an accessible environmental monitoring system that reduces dependence on manual observation. It collects sensor readings, displays them through a centralized dashboard and provides a local warning when a predefined temperature threshold is exceeded.

The system aims to answer the following questions:

- What are the current temperature and humidity levels?
- What analog reading is reported by the MQ-135 sensor?
- Is the high-temperature buzzer active?
- Is a valid GPS location available?
- How have sensor readings changed over time?
- Can collected data be reviewed and exported for further analysis?

Functional Scope

- Collection of temperature and humidity measurements using DHT22.
- Collection of analog gas-sensor readings using MQ-135.
- Reception of GPS information from the NEO-6M module.
- Activation of the buzzer when the temperature exceeds 34°C.
- Raw serial monitoring for troubleshooting.
- Report generation and CSV data export.
- Future integration of the PMS5003 particulate matter sensor after successful testing.

Technical Scope

- **Microcontroller:** ESP32 for sensor acquisition and processing.
- **Temperature and Humidity:** DHT22 sensor.
- **Gas Sensor:** MQ-135 analog output.
- **GPS Module:** NEO-6M for geographical coordinates.
- **Alert Device:** Buzzer for high-temperature warnings.
- **Firmware:** Arduino IDE and C++.
- **Communication:** UART for GPS data and USB serial communication with the web application.
- **Web Application:** Browser-based dashboard using the Web Serial API in a compatible browser.
- **Data Management:** Browser-side storage for historical records and CSV export.

Application Scope

The EcoGuard system may be useful in:

- **Indoor Environmental Monitoring:** Observing temperature, humidity and gas-sensor readings.
- **Air-Quality Observation:** Tracking changes in the MQ-135 sensor output.
- **Educational Projects:** Demonstrating IoT, embedded systems and sensor interfacing.
- **Laboratory Monitoring:** Recording environmental conditions in technical workspaces.

- **Environmental Data Collection:** Reviewing and comparing collected sensor readings.
- **Location-Based Monitoring:** Associating measurements with GPS coordinates during outdoor testing.
- **Future Air-Quality Expansion:** Adding particulate matter measurements through the PMS5003.

The system is a prototype for monitoring and educational purposes. MQ-135 analog readings are not calibrated gas concentrations or official air-quality index values.

System Overview

The EcoGuard system consists of five major layers.

Sensor Acquisition Layer

- DHT22 measures temperature and humidity.
- MQ-135 provides analog gas-sensor readings.
- NEO-6M supplies GPS information when a valid fix is available.
- The ESP32 collects the sensor data at regular intervals.

Embedded Processing and Alert Layer

- The ESP32 initializes the sensors and reads their values.
- GPS information is processed using the TinyGPSPlus library.
- The firmware compares the measured temperature with the 34°C threshold.
- The buzzer is activated when the threshold is exceeded and switched off when the condition clears.
- Sensor readings and status messages are transmitted through serial output.

Communication Layer

- The ESP32 sends readings through its USB serial connection.
- The web application receives data using a compatible browser's Web Serial interface.
- Incoming messages are parsed into temperature, humidity, gas, GPS and buzzer values.
- Invalid or unavailable readings are handled appropriately.

Web Dashboard and Data Management Layer

- Displays current sensor readings and available GPS coordinates.
- Shows alerts based on the hardware-reported buzzer status.
- Provides historical data review and raw serial monitoring.
- Supports report generation and CSV export.
- Can include simulation mode for testing without live hardware.

Testing and Future Expansion Layer

- Sensors are tested individually and as an integrated system.
- GPS is tested outdoors under suitable satellite visibility.
- Dashboard functions and data export are tested.
- PMS5003 integration remains pending until valid readings are obtained.

Advantages of the System

- **Real-Time Monitoring:** Displays available sensor readings as they arrive.
- **Integrated Design:** Combines temperature, humidity, gas-sensor and GPS information.
- **Automatic Warning:** Provides a local buzzer alert for high temperature.
- **Centralized Dashboard:** Presents measurements in one web interface.
- **Location Awareness:** Displays GPS coordinates when available.
- **Data Export:** Allows collected data to be exported for further analysis.
- **Troubleshooting:** Provides raw serial output for debugging.
- **Modular Design:** Supports future sensor integration.

Summary

The EcoGuard IoT Environmental Monitoring System uses an ESP32, DHT22, MQ-135, NEO-6M GPS module and buzzer to monitor environmental conditions. Sensor readings are transmitted to a web dashboard, where users can view current measurements, GPS information, hardware alerts and historical data.

The system also supports serial monitoring and data export, providing a practical foundation for IoT-based environmental observation. Future integration of the PMS5003 sensor can extend its particulate matter monitoring capabilities.

CHAPTER 2

LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

Recent research in environmental monitoring has increasingly focused on IoT-based sensing, real-time air-quality assessment, machine learning and environmental health analysis. Low-cost sensors connected to embedded systems allow environmental measurements to be collected continuously and presented through digital platforms.

Researchers have explored air-pollution monitoring systems, indoor air-quality management, particulate matter assessment and machine-learning-based forecasting. These approaches demonstrate how sensor networks and data-processing techniques can support environmental observation and early warnings. Some studies also investigate behavioural exposure patterns, personalized monitoring and environmental health risks.

The EcoGuard IoT Environmental Monitoring System combines an ESP32 microcontroller, DHT22 temperature and humidity sensor, MQ-135 gas sensor, NEO-6M GPS module, buzzer and web-based dashboard. The PMS5003 particulate matter sensor is considered a future extension after successful integration and testing.

2.2 LITERATURE SURVEY

Air Pollution Monitoring Systems

[1] S. Singh et al. presented a study titled *"Development of an Air Pollution Monitoring System"* in 2025. The research discusses air-pollution monitoring, sensor technologies and machine-learning techniques for air-quality forecasting. It highlights the importance of monitoring pollutant levels and understanding the factors contributing to air pollution. This study is relevant to EcoGuard because it supports the need for continuous environmental data collection. However, EcoGuard focuses on a compact, low-cost implementation with hardware alerts and a browser-based dashboard.

IoT-Based Indoor Air-Quality Management

[2] J. Rosa-Bilbao et al. proposed an IoT-based indoor air-quality management system for intelligent educational environments in 2025. The system monitors environmental parameters, including temperature, humidity and air pollutants, and incorporates machine-learning techniques for air-quality forecasting. The study demonstrates the value of combining connected sensors with real-time analysis. However, EcoGuard emphasizes an embedded prototype with GPS integration, buzzer-based alerts and direct serial communication with a web application.

Particulate Matter Distribution and Exposure Assessment

[3] C. Ou et al. investigated PM2.5 distribution and dynamic exposure assessment among university-community residents in 2024. Their work integrates low-cost sensors with an intelligent health-risk management system to examine environmental exposure across different scenarios. The study highlights the importance of particulate matter monitoring and location-dependent environmental assessment. EcoGuard can build on this concept by associating available GPS coordinates with environmental readings. Particulate matter measurement through the PMS5003 remains a planned extension of the current prototype.

Air-Quality Prediction Using Machine Learning

[4] M. M. Abdelmalek, H. Mahmoud and H. Shokry examined the prediction of air-quality index and air pollution using machine-learning techniques in 2025. Their work addresses the use of predictive models to estimate air-quality conditions from environmental data. Such approaches can support the identification of pollution trends and potential risks. However, EcoGuard currently concentrates on sensor acquisition, live visualization and threshold-based warnings rather than machine-learning-based forecasting.

Real-Time AIoT Air-Quality Monitoring

[5] A. Bekkar et al. developed a real-time AIoT platform for air-quality monitoring and prediction in Southwestern Morocco in 2024. The study combines low-cost sensors, network communication, feature selection and machine-learning models to predict PM2.5 concentrations. Its findings demonstrate the potential of integrating sensing and data analytics within an environmental monitoring platform. In comparison, EcoGuard focuses on a simpler embedded architecture and browser-based dashboard, with future opportunities for cloud connectivity and predictive analysis.

Personalized PM2.5 Forecasting

[6] C. Mountzouris, G. Protopsaltis and J. Gialelis investigated short-term PM2.5 forecasting using a low-cost wearable device and an attention-based LSTM model in 2025. The research explores how sensor data and temporal modelling can support personalized pollution forecasting. This work highlights the potential of combining environmental measurements with advanced analytical methods. EcoGuard provides a foundation for collecting environmental data, although personalized forecasting is outside its current implementation scope.

Real-Time IoT Air-Quality Forecasting

[7] O. Yildiz and H. S. Sucuoglu developed a real-time IoT-based air-quality forecasting system using machine learning in 2025. Their work integrates environmental sensors, embedded hardware, data processing and predictive models to estimate air-quality conditions. The study demonstrates how IoT sensing can be extended beyond monitoring to forecasting. EcoGuard currently prioritizes real-time sensor display and hardware alerts, while forecasting can be considered in future development.

Environmental Health-Risk Mapping

[8] M. Rajesh et al. presented a machine-learning-driven framework for real-time air-quality assessment and predictive environmental health-risk mapping in 2025. The research connects environmental monitoring with analytical methods for understanding potential health risks. It illustrates the value of converting sensor data into information that supports environmental assessment. EcoGuard offers a basic monitoring foundation but does not currently calculate predictive health-risk scores or official air-quality indices.

Personalized Air-Pollution Monitoring

[9] G. Navaneeth et al. proposed a personalized healthcare model for air-pollution monitoring and prediction using machine learning in 2025. The study explores the relationship between pollution monitoring, predictive analysis and personalized health applications. This research highlights the potential applications of environmental data in health-related decision support. EcoGuard focuses on general environmental observation and does not currently provide personalized healthcare predictions.

IoT Air-Quality Monitoring and Stress Analysis

[10] R. Manikandan et al. presented an AI-driven IoT framework for real-time air-quality monitoring and stress-correlation analysis in 2025. The work connects environmental monitoring with the analysis of possible relationships between air-quality conditions and stress-related factors. It illustrates the potential to extend sensor monitoring into broader environmental and human-wellbeing applications. EcoGuard currently concentrates on environmental measurements, GPS information and threshold-based alerts, leaving health-related correlation analysis for future research.

Comparison of Existing Techniques

Technique	Type	Strengths	Limitations
Conventional Monitoring	Manual measurement	Simple to perform	Limited continuous monitoring
IoT Sensor Monitoring	Embedded systems	Automated data collection	Requires reliable sensor integration
Indoor Air-Quality Management	IoT and analytics	Monitors multiple environmental parameters	May require more complex infrastructure
PM2.5 Monitoring	Particulate sensing	Supports particle concentration assessment	Requires a suitable particulate matter sensor
Machine-Learning Forecasting	Predictive analytics	Estimates future air-quality conditions	Requires suitable datasets and model validation
AIoT Monitoring	IoT and AI	Combines sensing with data analysis	Greater implementation complexity

GPS-Integrated Monitoring	Location-aware sensing	Associates readings with geographical locations	Requires a valid GPS fix
Threshold-Based Alerts	Embedded alert system	Provides immediate local warnings	Depends on appropriate thresholds and reliable measurements
Proposed EcoGuard System	Integrated IoT monitoring	Combines sensor readings, GPS, buzzer alerts, dashboard, history and data export	Calibration and PMS5003 integration require further work

Table 1: Comparison of Environmental Monitoring Techniques

2.3 REVIEW OF EXISTING IoT MONITORING SYSTEMS

Existing environmental monitoring systems demonstrate the benefits of integrating sensors with embedded controllers and data-processing platforms. Basic implementations collect measurements such as temperature, humidity and gas-sensor output, while advanced systems incorporate particulate matter monitoring, cloud connectivity and machine-learning-based prediction.

Recent studies also explore indoor air-quality management, personalized pollution forecasting and environmental health-risk assessment. These approaches show that environmental monitoring can extend beyond simple measurement to provide more detailed analysis and decision support.

EcoGuard combines the core functions of an environmental monitoring prototype through the ESP32, DHT22, MQ-135, NEO-6M GPS module and buzzer. The web dashboard provides centralized visualization, historical data review and data export. The system can be extended with particulate matter sensing and advanced analytics in future versions.

2.4 RESEARCH GAP

The reviewed studies identify several opportunities for developing an accessible environmental monitoring system:

- **Integrated Sensor Monitoring:** Combining temperature, humidity, gas-sensor readings and GPS information in one platform.
- **Real-Time Visualization:** Providing a centralized dashboard for displaying current sensor measurements.
- **Hardware-Based Alerts:** Providing a buzzer warning when the configured temperature threshold is exceeded.
- **Alert Synchronization:** Ensuring that the web application reflects the buzzer status reported by the ESP32.
- **Historical Data Management:** Supporting the review and export of collected environmental readings.

EcoGuard addresses these requirements by integrating environmental sensors, embedded processing, GPS, a buzzer and a web dashboard into a single prototype. Its main contribution is the practical integration of these components rather than the development of a new forecasting algorithm.

2.5 SUMMARY

The reviewed research demonstrates the growing role of IoT, low-cost sensors and machine learning in environmental monitoring. Existing studies explore air-pollution measurement, indoor air-quality management, PM2.5 assessment, real-time forecasting and environmental health-risk analysis. EcoGuard combines an ESP32, DHT22, MQ-135, NEO-6M GPS module and buzzer with a browser-based dashboard. It supports current sensor monitoring, available GPS coordinates, hardware-linked alerts, historical data review and data export. The PMS5003 remains a future extension until successful operation is verified.

Overall, the project provides a foundation for low-cost IoT environmental monitoring, with future opportunities for particulate matter sensing, sensor calibration, cloud connectivity and predictive analytics.

CHAPTER 3

SYSTEM DESIGN

3.1 Existing System

In the existing approach to environmental monitoring, temperature, humidity, and air quality are often measured using individual instruments or standalone sensor systems. The collected readings may need to be checked manually, making it difficult to monitor environmental changes continuously. In some systems, sensor values are displayed locally without providing a convenient way to view previous readings or manage alerts through a centralized interface. Traditional monitoring methods also have limitations in terms of portability, real-time data access, and location tracking. When environmental conditions change unexpectedly, users may not receive an immediate warning. In addition, data collected at different times may be difficult to compare if there is no proper mechanism for storing and organizing historical readings.

Some existing IoT monitoring systems provide online dashboards and automatic notifications, but they may require additional infrastructure, cloud services, or more complex configurations. The reliability of the collected information also depends on the quality, calibration, and correct installation of the sensors.

Therefore, an accessible environmental monitoring system should combine sensor measurements, real-time data display, alert generation, and historical data management in a single platform. It should also support location information where GPS signals are available and allow additional sensors to be integrated in the future.

3.2 Proposed System

The proposed system, **EcoGuard – IoT-Based Environmental Monitoring System**, combines an ESP32 microcontroller, environmental sensors, GPS, a buzzer, and a web-based dashboard to monitor selected environmental parameters. The main purpose of the system is to collect sensor readings, transmit them to a computer through serial communication, and present the information in a clear and user-friendly format.

The ESP32 acts as the central processing unit and receives readings from the connected sensors. A DHT22 sensor measures temperature and relative humidity, while the MQ-135 provides an

analog response related to the presence of certain gases and changes in air quality. The MQ-135 reading is treated as a sensor output rather than a calibrated gas concentration or official Air Quality Index (AQI), since accurate measurement requires suitable calibration and processing.

A NEO-6M GPS module is included to obtain geographical coordinates when a valid satellite fix is available. GPS performance may be limited indoors or in areas with poor satellite visibility. The system also uses a buzzer as a local warning mechanism. In the current firmware, the buzzer is activated when the measured temperature exceeds 34°C.

The ESP32 transmits sensor readings and status messages through its serial interface at 115200 baud. The web dashboard uses the Web Serial API in a compatible browser, such as Chrome or Edge, to receive and process the incoming data. The dashboard is designed to display current sensor values, maintain a history of readings, show alerts, and provide data export functionality. Where implemented and tested, the stored readings can be used to review changes over time and generate CSV reports.

The system is designed with future expansion in mind. A PMS5003 particulate matter sensor may be added to measure particulate matter such as PM2.5 and PM10, provided that its wiring, serial communication, and data parsing are successfully implemented and validated. Until then, the PMS5003 remains a planned extension rather than an operational component.

Objectives of the Proposed System

- To develop an IoT-based environmental monitoring system using the ESP32 microcontroller.
- To measure temperature and relative humidity using the DHT22 sensor.
- To collect analog gas-sensor readings from the MQ-135 module.
- To obtain GPS coordinates when the NEO-6M module receives a valid satellite fix.
- To activate a buzzer when the temperature exceeds the configured threshold of 34°C.
- To transmit sensor readings and device status messages to a web dashboard through serial communication.

- To present environmental readings in a clear and organized interface.
- To maintain historical readings and support CSV export through the dashboard.
- To provide a foundation for future particulate matter monitoring using the PMS5003 sensor.

3.3 Dataset Description

Unlike machine learning projects that depend on a fixed training dataset, EcoGuard primarily works with environmental readings collected directly from physical sensors. These readings form a time-based dataset that can be used to observe environmental conditions and examine how sensor values change over time.

The main data fields include temperature, relative humidity, MQ-135 analog readings, GPS latitude and longitude when available, and relevant device status messages. Depending on the serial output format and dashboard implementation, each reading may also be associated with a timestamp to help organize historical records.

The DHT22 provides temperature and humidity measurements. The MQ-135 produces an analog sensor value that may change with the surrounding gas conditions. However, this value should not be interpreted directly as a specific gas concentration or an official AQI without calibration and an appropriate conversion method. GPS coordinates are recorded only when valid location data is available. If the GPS module has not obtained a fix, the location should be treated as unavailable rather than as a confirmed position.

The collected information is transferred from the ESP32 to the web application through serial communication. The dashboard can use the received readings to display current values and, where implemented, maintain a history in browser storage and export records as CSV files.

The quality of the dataset depends on sensor condition, wiring, environmental factors, sampling frequency, and communication reliability. Missing GPS fixes, invalid sensor readings, or incomplete serial messages may affect the available records. Therefore, the incoming data should be checked before it is displayed or used for further analysis.

The PMS5003 is intended to extend the dataset with particulate matter measurements in a future version. Such readings should be included only after the sensor communicates correctly and its output has been validated.

3.4 System Architecture

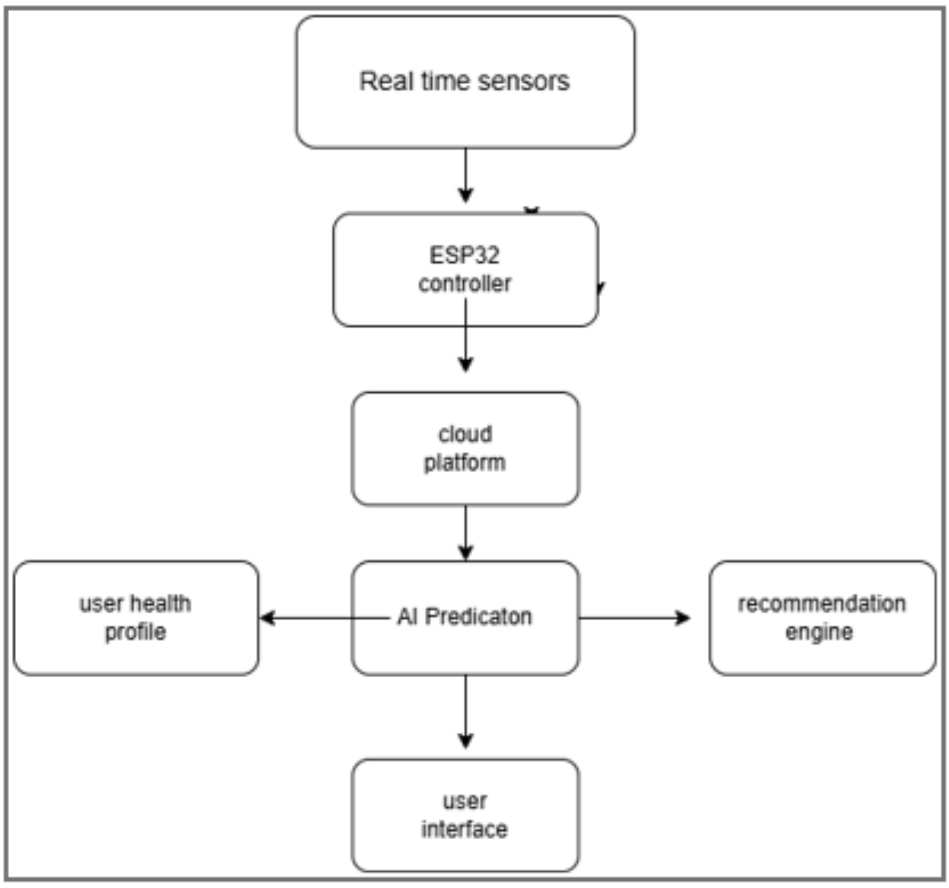


Figure 3.1: System Architecture of the EcoGuard IoT Environmental Monitoring System

The EcoGuard system is organized into four main layers: the sensor layer, the embedded processing layer, the communication layer, and the web dashboard layer. These layers work together to collect environmental information, process basic conditions, transmit the data, and present the results to the user.

1. Sensor Layer

This layer collects information from the surrounding environment through the connected hardware components.

- **DHT22 Sensor:** Measures temperature and relative humidity.
- **MQ-135 Sensor:** Provides an analog response related to the presence of certain gases and changes in air quality.
- **NEO-6M GPS Module:** Provides geographical coordinates when a valid GPS fix is available.
- **Buzzer:** Produces a local audible warning when the configured temperature condition is met.
- **PMS5003 (Future Extension):** Planned for particulate matter monitoring after successful integration and testing.

2. Embedded Processing Layer

The ESP32 serves as the central controller and coordinates the connected components.

- **Sensor Data Acquisition:** Reads temperature and humidity from the DHT22 and analog values from the MQ-135.
- **Threshold Evaluation:** Checks whether the measured temperature exceeds the configured threshold of 34°C.
- **Buzzer Control:** Activates or deactivates the buzzer according to the firmware's temperature-based alert logic.
- **Serial Output:** Formats sensor readings and status messages for transmission to the web application.

3. Communication Layer

This layer transfers the readings from the ESP32 to the computer running the dashboard.

- **Serial Communication:** Sends sensor values and status messages through the ESP32's USB serial connection.
- **Web Serial API:** Allows a supported browser to connect to the serial device after the user grants permission.
- **Data Parsing:** Processes incoming serial text and separates complete messages so that the dashboard can update the displayed values.
- **Status Synchronization:** Uses firmware status messages, such as buzzer ON and OFF notifications, to reflect the device's alert state in the dashboard.

4. Web Dashboard and Data Management Layer

This layer provides a user-friendly interface for viewing and managing the collected readings.

- **Real-Time Sensor Display:** Presents the latest available temperature, humidity, and MQ-135 readings.
- **GPS Information:** Displays location information when valid coordinates are received.
- **Alert Display:** Shows relevant warning information based on the status messages received from the firmware.
- **Sensor Details:** Provides additional information about individual sensors through the dashboard interface.

3.5 Hardware Connection Setup

The hardware setup connects the environmental sensors and alert components to the ESP32. Each module is connected to a suitable power supply and signal pin, with common ground where required. The voltage requirements of the modules must be checked before wiring to avoid damaging the ESP32 or the sensors.

Component	Pin or signal	ESP32 Connection
DHT22	Data	GPIO 4
MQ-135	Analog output	GPIO 34
Buzzer	Control signal	GPIO 25
NEO-6M GPS	GPS TX to ESP32 RX	GPIO 26
NEO-6M GPS	GPS RX from ESP32 TX	GPIO 27
NEO-6M GPS	UART baud rate	9600
ESP32	USB serial communication	Computer USB port
ESP32	Serial monitor baud rate	115200

Table 3.1: Hardware Pin Connections

The DHT22 data line is connected to GPIO 4, which is used by the firmware to read temperature and humidity. The analog output of the MQ-135 is connected to GPIO 34 so that the ESP32 can read the sensor signal using its analog-to-digital converter.

The buzzer control signal is connected to GPIO 25. The firmware uses this pin to control the local warning according to the configured temperature threshold. The NEO-6M GPS module communicates with the ESP32 through a UART connection. The GPS transmitter is connected to the ESP32 receiver pin at GPIO 26, while the GPS receiver is connected to the ESP32 transmitter pin at GPIO 27. The GPS baud rate is configured to 9600.

The ESP32 connects to the computer through USB, which carries the serial data used by the dashboard. The firmware's serial output is configured for 115200 baud, and the browser must use the same baud rate when opening the connection.

The exact power and signal wiring should be verified against the module specifications. In particular, the MQ-135 module's analog output must remain within the ESP32's permitted input voltage range. A voltage divider or suitable level-conditioning circuit may be required, depending

on the module and its supply voltage. GPS reception should also be tested outdoors or in an area with a clear view of the sky.

3.6 Software Requirements

EcoGuard uses embedded software to acquire sensor readings and a browser-based application to present the collected information. The ESP32 firmware is developed using the Arduino development environment, while the dashboard uses web technologies and the Web Serial API.

Software / Technology	Description
Operating System	Windows, Linux, or macOS
Embedded Controller	ESP32
Programming Environment	Arduino IDE or a compatible ESP32 development environment
Firmware Language	C/C++
DHT Sensor Library	Reads temperature and humidity from the DHT22
Adafruit Unified Sensor	Dependency used by the DHT sensor library
TinyGPSPlus	Parses GPS information from the NEO-6M module
Frontend Technologies	HTML, CSS, and JavaScript, according to the dashboard implementation
Serial Interface	Web Serial API
Supported Browser	Compatible desktop browser, such as Google Chrome or Microsoft Edge
Communication Baud Rate	115200 for ESP32-to-dashboard serial communication

GPS UART Baud Rate	9600
Data Storage	Browser localStorage, where implemented
Data Export	CSV file generation through the dashboard, where implemented
Development Environment	Visual Studio Code and/or Arduino IDE

Table 3.2: Software Requirements

3.7 Hardware Requirements

The following hardware is required to build and operate the current EcoGuard prototype. The minimum and recommended computer specifications are intended for running the development tools and web dashboard; they are not strict ESP32 hardware requirements.

Component	Minimum Requirement	Recommended Requirement
Microcontroller	ESP32 development board	ESP32 development board with USB connectivity
Temperature and Humidity Sensor	DHT22 module	DHT22 module with reliable connections
Gas Sensor	MQ-135 module	MQ-135 module with appropriate voltage conditioning
GPS Module	NEO-6M GPS module	NEO-6M with suitable antenna and clear-sky testing
Alert Component	Compatible buzzer	Buzzer with appropriate driver circuitry if required
Particulate Matter Sensor	Not required for the current prototype	PMS5003 after successful integration

Computer Processor	Intel Core i3 or equivalent	Intel Core i5 / AMD Ryzen 5 or equivalent
RAM	4 GB	8 GB or higher
Storage	2 GB free space	5 GB or more free space
USB Connection	Compatible USB data cable	Reliable USB data cable
Power Supply	Suitable supply for the modules	Stable, regulated supply meeting module requirements
Internet Connection	Not essential for basic local serial monitoring	Useful for development, updates, and online extensions

Table 3.3: Hardware Requirements

The ESP32 and its connected modules form the core hardware of the project. A computer is required to run the dashboard and establish the USB serial connection. A stable power supply and correct wiring are important for consistent sensor readings.

CHAPTER 4

IMPLEMENTATION

4.1 METHODOLOGY

The methodology describes the systematic approach followed in developing the EcoGuard IoT-Based Environmental Monitoring System. The main objective of the project is to monitor selected environmental parameters, display sensor readings in real time, and provide timely alerts when predefined conditions are detected. The system combines environmental sensors, an ESP32 microcontroller, GPS-based location information, serial communication, and a web-based dashboard in a single monitoring platform.

The system uses the ESP32 as its central controller. A DHT22 sensor measures temperature and relative humidity, while the MQ-135 provides an analog response associated with changes in the surrounding gas environment. A NEO-6M GPS module is used to obtain geographical coordinates when a valid satellite fix is available. A buzzer provides a local warning when the measured temperature exceeds the configured threshold of 34°C.

The collected readings are transmitted from the ESP32 to a computer through serial communication. The web dashboard receives this information using the Web Serial API in a compatible browser. The dashboard is designed to present the latest sensor values, show relevant alerts, and support the review of historical readings and CSV data exports where these features are implemented.

The system follows a modular design in which sensor acquisition, embedded processing, GPS parsing, buzzer control, serial communication, and dashboard operations are handled as separate functions. This approach makes the system easier to develop, test, and maintain. It also allows additional sensors to be introduced without requiring a complete redesign of the existing system.

The current implementation focuses on collecting and presenting sensor readings rather than performing artificial intelligence-based environmental prediction. Future versions may incorporate particulate matter measurements and predictive analysis after the required sensors, datasets, and algorithms have been successfully integrated and validated.

Research Approach

The project follows a hardware-and-software development approach that combines embedded systems, environmental sensing, serial communication, web development, and system testing. Development is carried out in stages so that individual components can be checked before being integrated into the complete monitoring system.

The research approach focuses on developing a practical system that can collect environmental measurements, communicate the readings to a dashboard, and notify users when a configured threshold is exceeded.

The major development activities are described below:

1. **Requirement Analysis:** The functional requirements are identified, including temperature and humidity measurement, gas-sensor reading acquisition, GPS information, buzzer alerts, serial communication, dashboard display, and historical data management.
2. **Hardware Selection and Setup:** The ESP32, DHT22, MQ-135, NEO-6M GPS module, and buzzer are selected and connected according to their electrical and communication requirements.
3. **Firmware Development:** The ESP32 firmware is developed to initialize the hardware, read sensor values, process GPS data, check the temperature threshold, control the buzzer, and transmit status messages.
4. **Sensor Testing:** Each sensor is tested independently to confirm that the hardware connections and corresponding firmware functions operate as expected.
5. **Serial Communication:** The ESP32 transmits readings through its USB serial connection at 115200 baud. The GPS module communicates with the controller through UART at 9600 baud.
6. **Dashboard Development:** A browser-based dashboard is developed to receive and display the serial data. The interface organizes sensor readings and provides relevant monitoring and data-management features.
7. **Integrated System Testing:** The complete system is tested to verify the interaction between sensors, the ESP32, the serial interface, and the dashboard.
8. **Future Expansion Planning:** The PMS5003 particulate matter sensor is considered for a future version. Its data should be included only after communication, parsing, and measurement behaviour have been verified.

This research approach supports step-by-step development and helps identify hardware, communication, and software issues before the system is evaluated as a complete unit.

System Development Methodology

The system is developed through a series of interconnected stages. Each stage performs a specific function and provides the necessary output for the next stage.

1. Sensor Data Acquisition

The DHT22 provides temperature and relative humidity readings, while the MQ-135 produces an analog sensor value. These readings are acquired by the ESP32 using the configured digital and

analog input pins. The NEO-6M GPS module provides location information when satellite reception is sufficient.

2. Embedded Data Processing

The ESP32 processes the incoming sensor information and prepares the data for transmission. The firmware checks the measured temperature against the configured threshold of 34°C. When the threshold is exceeded, the buzzer is activated according to the implemented control logic.

3. GPS Data Processing

GPS information is received through the UART interface and processed using the TinyGPSPlus library. Valid latitude and longitude values can be passed to the dashboard. When a valid GPS fix is unavailable, the system should indicate that location information is not currently available.

4. Serial Communication

The ESP32 sends sensor readings and status messages through the USB serial connection. The firmware uses a baud rate of 115200 for communication with the dashboard, while the GPS module uses a separate UART connection configured at 9600 baud.

5. Dashboard Processing and Visualization

The web application receives serial data using the Web Serial API. It processes complete incoming messages and updates the corresponding sensor displays. Depending on the implemented features, the dashboard can also maintain historical records, show alert information, and export available readings to CSV files.

6. Alert Management

The physical buzzer is controlled by the ESP32 firmware. The dashboard reflects the alert state using the status messages received from the controller. This keeps the displayed alert connected to the actual firmware status instead of relying only on the temperature value shown on the screen.

7. Testing and Validation

The final stage checks the sensor readings, threshold behaviour, GPS availability, serial communication, dashboard updates, and data management features. The results are reviewed to identify limitations and areas for improvement.

The methodology establishes a clear flow of information from the physical environment to the web interface. It also keeps the hardware-controlled alert mechanism separate from the dashboard's visual notification system.

Data Collection and Processing

EcoGuard collects environmental information directly from connected hardware rather than relying on a fixed machine learning training dataset. The readings can be organized into a time-based record to support the observation of environmental changes during the monitoring period.

The primary data fields include temperature, relative humidity, MQ-135 analog readings, GPS coordinates when available, and relevant device status messages. Timestamps may also be associated with the readings to support chronological review and comparison.

The data collection process includes the following activities:

1. **Temperature and Humidity Collection:** The DHT22 measures the surrounding temperature and relative humidity.
2. **Gas-Sensor Data Collection:** The ESP32 reads the analog output from the MQ-135. The raw reading indicates the sensor's response and should not be treated directly as a calibrated gas concentration or official AQI.
3. **Location Data Collection:** The NEO-6M provides GPS coordinates when a valid fix is obtained. Indoor reception and obstructed views of the sky may delay or prevent a fix.
4. **Data Transmission:** The ESP32 sends sensor values and status messages to the computer through serial communication.
5. **Data Parsing:** The dashboard processes incoming serial messages and extracts the values required for display.
6. **Historical Data Management:** Where implemented, readings are organized and retained in browser storage for later review.
7. **Data Export:** Available records can be exported as CSV files when the dashboard's report-generation feature is enabled.

Particulate matter measurements such as $PM_{2.5}$ and PM_{10} are planned for future expansion through the PMS5003 sensor. Until the module is integrated and tested successfully, these values must not be represented as actual measurements collected by the current EcoGuard prototype.

4.2 IMPLEMENTATION

The implementation of EcoGuard is divided into interconnected hardware and software modules. The hardware layer collects environmental measurements, the ESP32 firmware processes and transmits the readings, and the web dashboard presents the information to the user.

The firmware is developed in C/C++ using the Arduino development environment or a compatible ESP32 toolchain. The DHT sensor library is used to read temperature and humidity, the Adafruit Unified Sensor library provides a dependency for sensor support, and TinyGPSPlus processes GPS information.

The dashboard uses web technologies and the Web Serial API to receive data from the ESP32. The serial connection is configured at 115200 baud to match the firmware. The interface organizes the received values into sensor displays and supports additional monitoring features according to the current implementation.

The implementation follows a sequential workflow. First, the hardware and communication interfaces are initialized. Next, the firmware reads the available sensor values and checks the temperature threshold. The readings and status messages are then transmitted to the dashboard, which parses the incoming information and updates the display.

This modular structure allows individual components to be tested independently and makes future expansion more manageable.

4.2.1 Sensor Data Acquisition Module

Purpose

The sensor data acquisition module collects the environmental readings required by EcoGuard. It provides the input information used by the embedded processing and dashboard modules.

Implementation

The DHT22 is connected to GPIO 4 and is used to measure temperature and relative humidity. The MQ-135 analog output is connected to GPIO 34, allowing the ESP32 to read the sensor response through its analog-to-digital converter.

The firmware reads the available values and prepares them for further processing. Sensor errors or unavailable readings should be handled without presenting invalid measurements as reliable data.

Implementation Steps

1. Initialize the ESP32 and connected sensor interfaces.
2. Configure GPIO 4 for DHT22 communication.
3. Configure GPIO 34 as the MQ-135 analog input.
4. Read temperature and humidity from the DHT22.
5. Read the analog output of the MQ-135.
6. Check the validity of the readings where supported.

7. Prepare the collected values for threshold checking and serial transmission.

Output

The module provides temperature, relative humidity, and MQ-135 analog readings to the firmware's processing and communication functions.

4.2.2 GPS Integration Module

Purpose

The GPS integration module obtains geographical information that can help associate environmental observations with a location.

Implementation

The NEO-6M GPS module communicates with the ESP32 through UART. The GPS transmitter is connected to ESP32 GPIO 26, and the GPS receiver is connected to GPIO 27. The GPS communication baud rate is configured at 9600.

The TinyGPSPlus library processes the received GPS data and extracts available information, including latitude and longitude. A valid location is available only when the GPS module obtains a suitable satellite fix. The time required to acquire a fix depends on the environment, satellite visibility, antenna placement, and receiver conditions.

Implementation Steps

1. Initialize the GPS UART interface at 9600 baud.
2. Receive the data transmitted by the GPS module.
3. Pass incoming characters to the TinyGPSPlus parser.
4. Check whether the location information is valid.
5. Provide latitude and longitude when a valid fix is available.
6. Indicate unavailable location data when the module has not obtained a valid fix.

Output

The module provides geographical coordinates when available. These coordinates can be displayed with the sensor readings to help users understand where an observation was recorded.

4.2.3 Buzzer-Based Alert Module

Purpose

The buzzer-based alert module provides an audible warning when the measured temperature exceeds the configured threshold.

Implementation

The buzzer control signal is connected to GPIO 25. The ESP32 firmware compares the temperature reading against the threshold of 34°C. When the temperature exceeds this value, the firmware activates the buzzer and sends a corresponding status message. When the condition is no longer met, the firmware deactivates the buzzer according to its implemented logic.

The dashboard uses the received buzzer status messages to represent the device's alert state. For example, the message BUZZER: ON - HIGH TEMPERATURE indicates that the firmware has activated the high-temperature alert. A corresponding OFF message clears the active alert state.

The physical buzzer and the dashboard notification are related but separate functions. Acknowledging an alert in the web application does not itself switch off the physical buzzer; the buzzer is controlled by the ESP32 firmware.

Implementation Steps

1. Read the current temperature from the DHT22.
2. Compare the reading with the configured threshold of 34°C.
3. Activate the buzzer when the threshold condition is met.
4. Transmit the corresponding alert status through serial communication.
5. Deactivate the buzzer when the condition is no longer met, according to the firmware logic.
6. Update the dashboard alert display based on the received status messages.

Output

The module provides a local audible warning and a corresponding status message for the dashboard when the configured high-temperature condition occurs.

4.2.4 Serial Communication Module

Purpose

The serial communication module transfers sensor readings and device status messages from the ESP32 to the web dashboard.

Implementation

The ESP32 uses its USB serial interface to transmit the firmware output at 115200 baud. The dashboard connects through the browser's Web Serial API after the user selects and grants permission to the connected serial device.

Incoming data may arrive in separate chunks rather than as complete messages. The dashboard therefore needs to buffer incoming text and process complete lines before extracting the sensor values. This reduces the risk of interpreting incomplete messages as valid readings.

The serial message format must remain consistent between the firmware and dashboard. If the firmware changes the labels or arrangement of the transmitted values, the dashboard's parsing logic may also need to be updated.

Implementation Steps

1. Initialize the ESP32 serial interface at 115200 baud.
2. Format sensor readings and device status messages.
3. Transmit the formatted information to the computer.
4. Open the serial connection through a compatible browser.
5. Read incoming data and buffer incomplete chunks.
6. Process complete lines and extract the required values.
7. Pass the parsed information to the dashboard display.

Output

The module provides a continuous data path between the ESP32 and the dashboard while allowing sensor readings and device status messages to be updated as they arrive.

4.2.5 Real-Time Dashboard Module

Purpose

The dashboard provides a user-friendly interface for viewing the environmental readings received from the ESP32.

Implementation

The browser-based dashboard connects to the ESP32 using the Web Serial API. Once the user authorizes the serial device, the application receives and parses the transmitted readings. The sensor cards display the latest available values, while relevant status information is used to represent alerts and device conditions.

The dashboard may also provide sensor-specific details, historical readings, a raw serial monitor, and report-generation features, depending on the current implementation.

Main Features

- Display temperature and relative humidity.
- Display the analog MQ-135 sensor reading.
- Present GPS coordinates when a valid fix is received.
- Reflect the buzzer status using firmware messages.
- View previous readings where history storage is enabled.
- Export available historical records as CSV files.
- Inspect incoming serial messages through the raw serial monitor.
- Display the integration status of planned sensors such as the PMS5003.

Output

The module presents the available environmental information in an organized interface, making it easier to monitor readings and inspect the device's status.

4.2.6 Historical Data and Report Generation Module

Purpose

This module supports the review and export of previously collected environmental readings.

Implementation

Where implemented, the dashboard maintains a history of received readings and stores the relevant records in browser localStorage. The historical data can be organized for review over time, while the CSV export function allows the user to save available records for further analysis.

The usefulness of the historical record depends on the data received during operation. Missing readings, browser storage limitations, and interruptions in serial communication may affect the completeness of the stored data.

Implementation Steps

1. Receive and parse a valid sensor message.
2. Associate the reading with a timestamp where supported.
3. Add the reading to the dashboard's history.
4. Store the record in browser storage where enabled.
5. Retrieve the available records for historical review.

6. Convert the selected records into CSV format when the export function is available.

Output

The module provides access to previously collected readings and supports offline review of exported records.

4.2.7 PMS5003 Future Integration Module

Purpose

The planned PMS5003 module is intended to extend EcoGuard with particulate matter monitoring, including $PM_{2.5}$ and PM_{10} measurements.

Implementation Status

The PMS5003 is a future extension and is not treated as an operational sensor in the current prototype. Successful integration requires correct power connections, UART configuration, valid frame parsing, and verification of the reported particulate matter values.

Planned Implementation Steps

1. Verify the module's power and interface requirements.
2. Connect the PMS5003 to a suitable UART interface.
3. Implement a parser for its documented data frame format.
4. Check frame validity and handle communication errors.
5. Compare the received values with suitable reference measurements where possible.
6. Add validated $PM_{2.5}$ and PM_{10} values to the dashboard.
7. Store and export the new readings alongside existing environmental data.

Expected Output

After successful integration and testing, the module may provide particulate matter readings for historical analysis and future environmental risk assessment. Until then, $PM_{2.5}$ and PM_{10} values must be treated as unavailable in the actual system.

4.3 TESTING AND VALIDATION

Testing and validation are required to verify the operation of the individual hardware components and the complete EcoGuard system. The testing process focuses on sensor readings, GPS reception, buzzer behaviour, serial communication, dashboard updates, and historical data functions.

4.3.1 Testing Phases

GPS Testing

The GPS module is tested in an outdoor area with a clear view of the sky. The availability of a valid fix and the display of coordinates are checked. Indoor operation is also considered when documenting GPS limitations.

Alert Testing

The firmware is checked to verify that the buzzer activates when the temperature exceeds 34°C and changes state according to the implemented threshold logic. The dashboard's alert display is checked against the corresponding serial status messages.

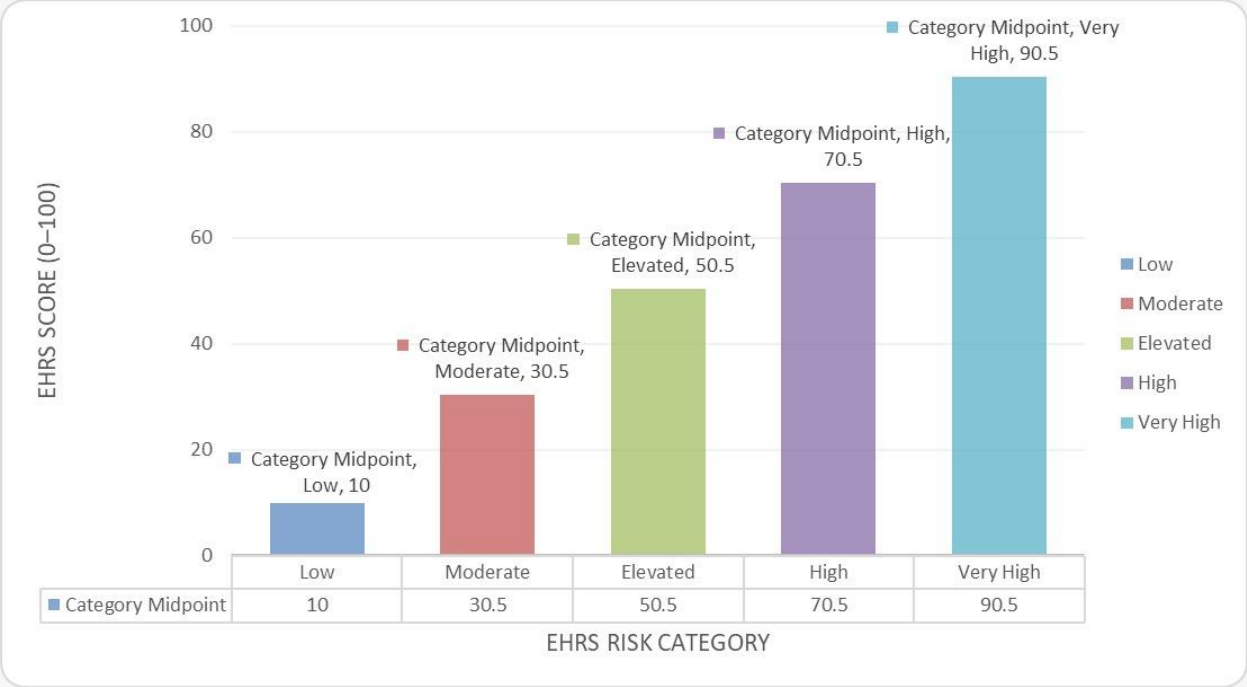


Figure 4.1: EHRS Risk Category

4.3.2 Functional Test Cases

Test Case	Test Procedure	Expected Result
DHT22 Reading	Observe temperature and humidity output	Valid temperature and humidity values are received
MQ-135 Reading	Observe the analog sensor output	An analog reading is received and displayed

GPS Reception	Test outdoors with clear sky visibility	Coordinates appear when a valid GPS fix is obtained
Temperature Alert	Raise the measured temperature above 34°C under controlled conditions	Firmware activates the buzzer and transmits the ON status
Alert Clearing	Allow the temperature to return below the threshold	Firmware clears the alert according to its logic
Serial Communication	Connect the ESP32 to a supported browser	Incoming serial messages are received and parsed
Dashboard Update	Observe the interface while new readings arrive	Available sensor values update correctly
History and CSV	Use the implemented history and export functions	Available readings can be reviewed and exported
PMS5003	Validate after future integration	Valid particulate matter readings are parsed and checked

Table 4: Functional Test Cases for EcoGuard

The table describes the expected outcomes of the tests. Actual results should be recorded after the corresponding tests have been performed. A test should not be marked as passed solely because the expected result has been defined.

4.3.3 Environmental Data Visualization

Environmental data visualization helps users understand changes in environmental conditions over a period of time. The EcoGuard prototype can use time-series graphs to represent sensor readings, making it easier to observe trends, compare values, and identify sudden variations.

The graph uses time on the horizontal axis and the corresponding sensor readings on the vertical axis. Temperature and humidity readings help monitor changes in surrounding environmental conditions, while MQ-135 readings indicate variations in the sensor's response to certain gases. These readings should be interpreted according to sensor characteristics and calibration status.

For the current prototype, graphs can be generated using actual temperature, humidity, and MQ-135 readings collected during system operation. $PM_{2.5}$ and PM_{10} measurements can be included only after the PMS5003 sensor has been successfully integrated and its readings validated.

Overall, environmental data visualization improves the readability of sensor data and supports trend analysis and future enhancements, such as historical monitoring and threshold-based alerts.

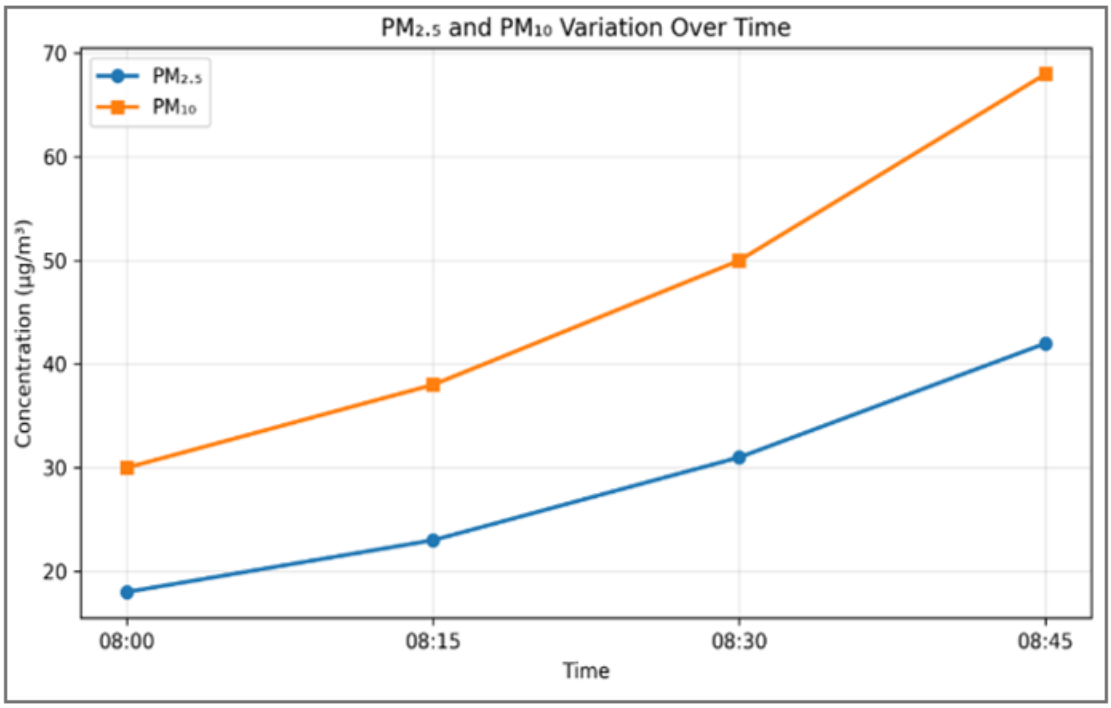


Figure 4.2: Time Vs Environmental Sensor Readings

Suggested graph description: The graph plots the measured environmental parameters against their corresponding timestamps. The horizontal axis represents the observation time, while the vertical axis represents the measurement scale of each parameter. If different units or scales are used, separate axes or individual graphs should be considered to avoid misleading comparisons.

A rising MQ-135 value indicates a change in the sensor's analog response, but it does not by itself establish a specific gas concentration or confirm a proportional change in particulate matter. Similarly, temperature and humidity variations should be interpreted using their own measurement units.

4.3.4 Results and Discussion

The results of the EcoGuard project should be discussed in terms of the functions verified during testing. The primary objective is to demonstrate that the ESP32 can acquire the available sensor readings, transmit them through serial communication, and display them through the dashboard.

The DHT22 is intended to provide temperature and humidity information, while the MQ-135 supplies an analog response that can be monitored over time. The GPS module adds location information when a valid fix is obtained. The buzzer provides a local warning when the configured temperature condition is met.

The reliability of the results depends on the accuracy and condition of the sensors, correct electrical connections, stable communication, and appropriate interpretation of the readings. The MQ-135 requires suitable calibration before its output can be interpreted quantitatively. GPS coordinates may be unavailable indoors, and the PMS5003 remains a planned extension until successful integration.

The final report should include measured test results, screenshots of the working dashboard, and actual sensor observations collected during the experiment. Quantitative claims about accuracy, particulate matter concentration, or prediction performance should be included only when supported by suitable measurements and evaluation.

4.4 ETHICAL AND SAFETY CONSIDERATIONS

The EcoGuard system is designed to assist environmental observation. Its measurements should be interpreted carefully, particularly when users may rely on them to assess environmental conditions.

Sensor Accuracy: Low-cost sensors may be affected by temperature, humidity, interference, warm-up time, and other environmental factors. Readings should not be treated as laboratory-grade measurements without appropriate calibration and validation.

Electrical Safety: The power and signal requirements of the connected modules must be checked before wiring. The ESP32 input voltage limits must be respected, especially when connecting analog outputs from modules operating at higher supply voltages.

GPS Limitations: Location information should be displayed only when valid data is available. Indoor reception and obstructed satellite visibility may affect GPS performance.

Alert Reliability: The buzzer threshold is a configured project setting, not a universal environmental safety standard. Users should not treat the alert as a replacement for certified monitoring equipment or official safety guidance.

Data Integrity: Historical readings should be associated with accurate timestamps where possible. Missing or invalid measurements should be identified rather than silently represented as valid data.

Privacy: If location information or environmental records can reveal sensitive details about a monitoring site, access to stored data and exported reports should be managed appropriately.

Future AI Integration: Any future prediction model or environmental risk score should be evaluated using suitable data and documented methods. Recommendations should communicate their limitations and should not be presented as validated health advice without appropriate evidence.

4.5 SUMMARY

This chapter presented the methodology, implementation, and testing approach of the EcoGuard IoT-Based Environmental Monitoring System. The system integrates the ESP32 microcontroller with the DHT22 temperature and humidity sensor, MQ-135 analog gas sensor, NEO-6M GPS module, and a buzzer-based alert mechanism to monitor environmental conditions and provide alerts when configured thresholds are exceeded.

The implementation focuses on collecting environmental readings, processing sensor data, evaluating the configured temperature threshold, and transmitting measurements and system status messages through serial communication. The collected readings can be used to observe variations in environmental conditions and support further analysis. Environmental data visualization also provides a structured way to understand changes in sensor values over time.

The testing plan covers individual sensor operation, GPS signal reception, buzzer control, serial communication, dashboard functionality, and data management. These tests help verify the operation of the individual components and the overall system. Actual test results must be recorded after testing, and the reliability of the measurements should be evaluated before drawing conclusions. The limitations of low-cost sensors, including calibration requirements and variations in measurement accuracy, have also been considered.

Furthermore, the PMS5003 particulate-matter sensor and AI-based environmental risk prediction are identified as future extensions rather than completed functionalities. Their integration would enable the system to monitor $PM_{2.5}$ and PM_{10} concentrations and potentially provide more advanced environmental risk assessments after suitable validation.

Overall, this chapter establishes the technical foundation of the EcoGuard prototype, describes its implementation and verification procedures, and identifies areas for improvement. The approach ensures that the project report accurately reflects the current system capabilities while providing a clear direction for future development and more comprehensive environmental monitoring.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Testing Methodology

The proposed IoT-based environmental health monitoring system was tested in individual and integrated stages to verify the functionality of its sensors, ESP32 microcontroller, web dashboard, GPS module, and buzzer alert mechanism. Each sensor was initially tested independently to confirm that it produced readable output before integrating the components into a single system.

After individual testing, the sensors were connected to the ESP32 and their readings were monitored through the Arduino Serial Monitor. The web dashboard was then evaluated to verify sensor data display, historical readings, report generation, and alert status updates. The buzzer was tested by checking its response to the temperature threshold configured in the ESP32 firmware. GPS functionality was evaluated by checking satellite availability and location-fix status.

The system was tested under available environmental conditions. Since GPS reception depends on satellite visibility, indoor testing was also used to observe the behaviour of the GPS module when a valid fix was unavailable.

5.2 Individual Sensor Testing

Each sensor was tested separately before integration to identify connection, power supply, and data acquisition issues.

- DHT22 sensor: The sensor successfully returned temperature and relative humidity readings. During testing, a temperature reading of approximately 34.2°C and humidity of 72% was observed. These values demonstrate successful data acquisition under the test conditions.
- MQ-135 gas sensor: The sensor returned varying analog values through the ESP32 ADC. Readings such as 560–608 were observed during testing. These values represent raw analog measurements and are not direct gas concentrations in parts per million (PPM).
- NEO-6M GPS module: The module transmitted GPS data, but the indoor test displayed zero satellites and a “NO FIX” status. This indicates that a valid position was not established during that test.
- Buzzer: The buzzer was verified independently and subsequently tested with the ESP32 temperature-based control logic.
- PMS5003 particulate matter sensor: Initial communication testing did not establish consistently verified particulate-matter measurements. Therefore, its data is excluded from the confirmed sensor results and further testing is required.

The individual testing stage helped identify which components were functioning correctly and which required additional investigation.

5.3 Integrated Hardware Testing

After individual testing, the DHT22, MQ-135, NEO-6M GPS module, and buzzer were integrated with the ESP32. The DHT22 was connected to GPIO 4, the MQ-135 analog output to GPIO 34, and the buzzer control to GPIO 25. The GPS module communicated with the ESP32 through a hardware serial interface using GPIO 26 and GPIO 27 for RX and TX, respectively.

The integrated program collected sensor readings and displayed them through the Serial Monitor at approximately two-second intervals. Temperature, humidity, and raw gas-sensor values were obtained during testing. GPS information was displayed according to the availability of a valid location fix, and the buzzer state was reported according to the programmed temperature threshold.

The results demonstrated that the ESP32 could collect readings from multiple connected components and execute the programmed alert logic. However, the reliability of each sensor depends on proper wiring, calibration, power supply, and environmental conditions.

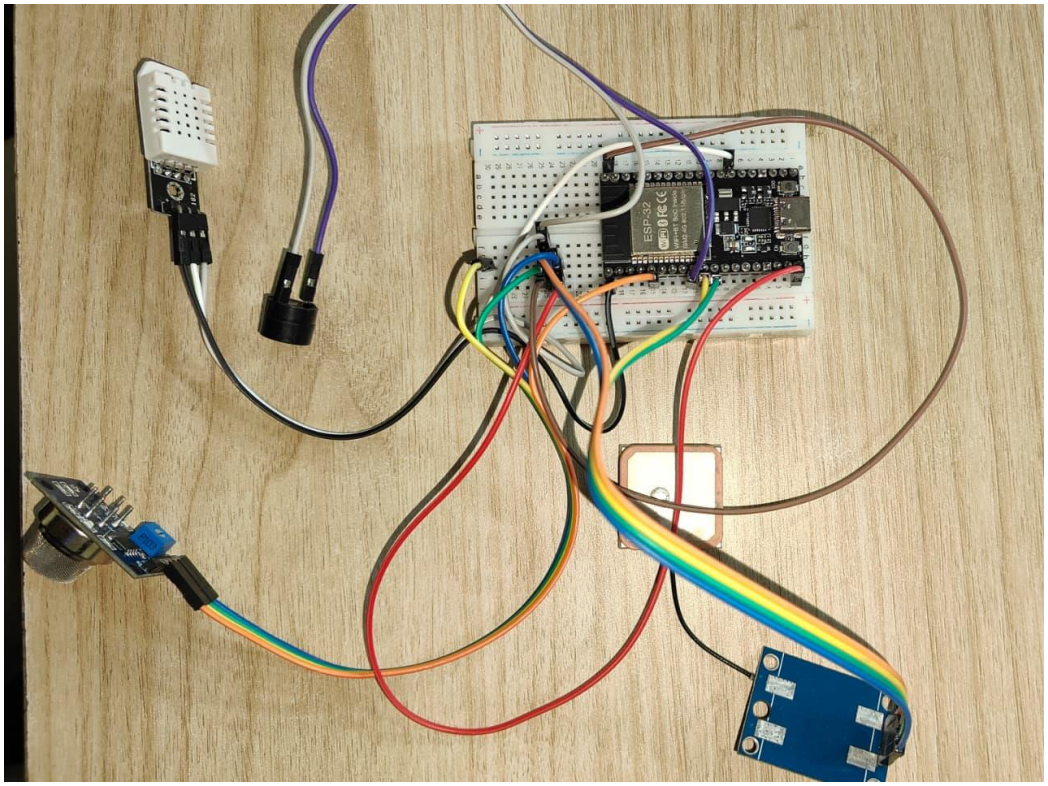


Figure 5.1. Integrated Hardware Setup

Insert a photograph of your actual ESP32 board with the connected DHT22, MQ-135, GPS module, and buzzer here. Use a small, clear image centred on the page.

5.4 Dashboard Functionality Testing

The EcoGuard web dashboard was designed to provide a consolidated view of the environmental sensor readings and system status. It presents the available temperature, humidity, MQ-135 analog value, GPS fix status, satellite count, and buzzer state.

The dashboard functionality was evaluated by checking whether the received sensor data could be displayed in the corresponding interface components. The navigation sections for Dashboard, History, Reports, Sensors, Alerts, and Settings were designed to support monitoring, reviewing stored readings, and inspecting system information.

The alert interface follows the ESP32 firmware output. When the serial message BUZZER: ON - HIGH TEMPERATURE is received, the dashboard displays the corresponding active alert. When BUZZER: OFF is received, the active alert is cleared. A temperature value above the threshold alone does not independently trigger a dashboard alert.

Historical data and reports can help users review changes over time, provided that readings are successfully received and stored. Dashboard screenshots should be included to demonstrate the interface and the sensor values displayed during the test.

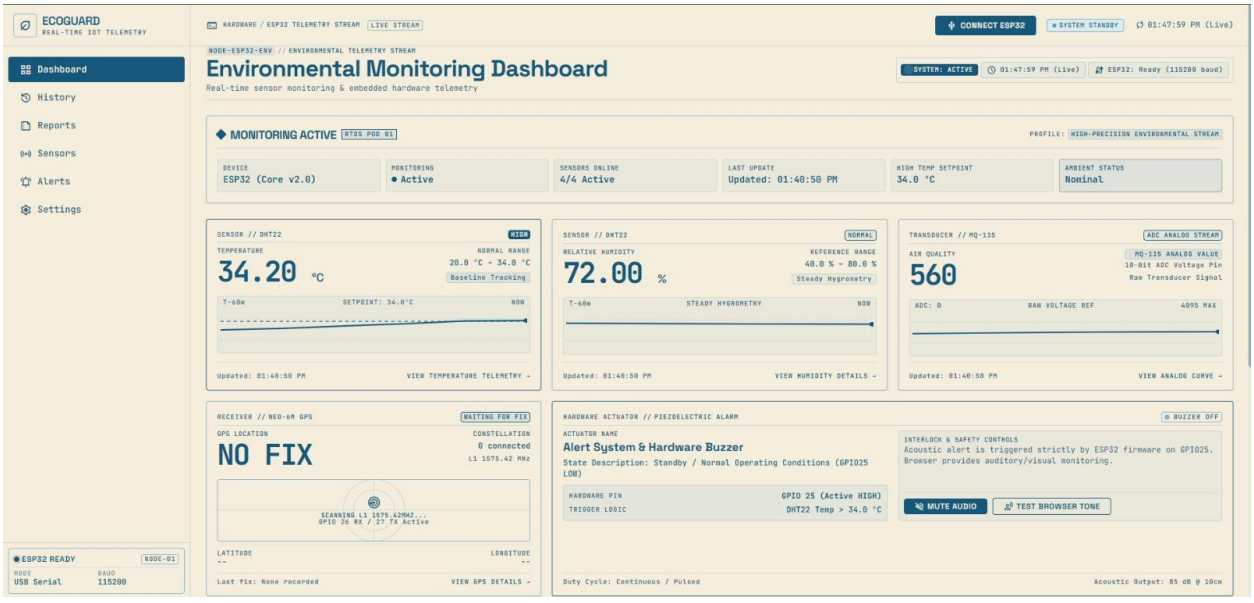


Figure 5.2. EcoGuard Environmental Monitoring Dashboard

Insert a screenshot of your actual dashboard showing the sensor cards, connection status, and navigation tabs. If available, use a screenshot with live sensor values rather than simulated readings.

5.5 Buzzer Alert Testing

The buzzer alert mechanism was tested using the temperature threshold defined in the ESP32 program. The firmware activates the buzzer when the DHT22 returns a valid temperature greater than 34°C. Otherwise, the buzzer is switched off.

During testing, a temperature reading of approximately 34.2°C resulted in the output BUZZER: ON - HIGH TEMPERATURE. This confirmed that the programmed threshold condition was reached and the firmware issued the buzzer activation command.

The dashboard alert mechanism was designed to reflect the buzzer state reported by the ESP32. The alert remains active while the firmware reports the buzzer as ON and is cleared when the firmware reports it as OFF. This approach keeps the web interface aligned with the hardware control logic.

The current implementation uses a fixed threshold and is intended for system demonstration. Further testing under different temperature conditions is necessary to evaluate the consistency of the buzzer response.

5.6 GPS Fix and Location Testing

The NEO-6M GPS module was tested by receiving serial data through the ESP32 hardware serial interface. The program used the TinyGPS++ library to decode GPS messages and obtain latitude, longitude, and satellite information when available.

During indoor testing, the Serial Monitor displayed GPS: NO FIX and Satellites: 0. This result indicates that the module did not establish a valid position fix under the test conditions. A GPS module may receive serial messages while still being unable to calculate a valid geographic location.

5.7 Results and Discussion

The testing process demonstrated the feasibility of integrating environmental sensors, an ESP32 microcontroller, a GPS module, and a buzzer into a unified monitoring system. The DHT22 provided temperature and humidity measurements, while the MQ-135 supplied raw analog readings for monitoring changes in its sensor output. The buzzer logic responded to the programmed temperature threshold, and the serial output provided a useful method for verifying the behaviour of the integrated firmware.

The GPS module transmitted data, but a valid location fix was not obtained during the reported indoor test. Similarly, the PMS5003 requires further investigation before particulate-matter measurements can be included in the confirmed results. These limitations identify areas for future improvement rather than indicating that all components have been fully validated.

The web dashboard provides a convenient interface for presenting sensor information, monitoring buzzer status, and reviewing readings. Its usefulness depends on reliable serial communication, accurate parsing, and appropriate data storage. The raw MQ-135 values should not be interpreted as calibrated air-quality concentrations without suitable calibration and validation.

Overall, the results support the basic operation of the tested monitoring components and the feasibility of the proposed system architecture. Additional outdoor GPS testing, PMS5003 communication testing, sensor calibration, and longer-duration integrated trials are recommended to improve reliability and establish performance under a wider range of conditions.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The project titled **"EcoGuard – IoT-Based Environmental Monitoring System"** was developed to monitor environmental conditions using IoT technology. The system combines an ESP32 microcontroller, DHT22 temperature and humidity sensor, MQ-135 gas sensor, NEO-6M GPS module, buzzer, and a web-based dashboard to provide accessible environmental monitoring.

The ESP32 collects sensor readings and transmits them to the dashboard through serial communication. The DHT22 measures temperature and humidity, while the MQ-135 provides an analog response related to changes in the surrounding gas environment. The GPS module provides location information when a valid satellite fix is available. A buzzer is activated when the temperature exceeds the configured threshold of 34°C.

The web dashboard presents sensor readings and device status in an organized interface. It is designed to support real-time monitoring, historical data review, alerts, and CSV data export. These features help users observe environmental changes and review previously collected measurements.

The project demonstrates how low-cost hardware and web technologies can be combined to develop a practical environmental monitoring prototype. However, sensor accuracy depends on proper connections, calibration, and operating conditions. The MQ-135 reading should not be treated as a calibrated gas concentration or official AQI without appropriate calibration.

Overall, EcoGuard provides a foundation for accessible environmental monitoring and future expansion. Integration of the PMS5003 particulate matter sensor, followed by suitable testing and validation, could extend the system to $PM_{2.5}$ and PM_{10} monitoring. Additional data analysis and prediction features may also be introduced in future versions.

Purpose and Achievements

The main purpose of EcoGuard is to collect environmental readings, present them through a web dashboard, and provide a warning when a predefined temperature condition is detected. The major achievements are summarized below.

1. **IoT-Based Monitoring:** Developed a monitoring prototype using the ESP32 and connected environmental sensors.
2. **Temperature and Humidity Measurement:** Integrated the DHT22 to collect temperature and relative humidity readings.

3. **Gas-Sensor Integration:** Connected the MQ-135 to collect analog sensor readings related to changes in the surrounding environment.
4. **GPS Integration:** Incorporated the NEO-6M module to obtain geographical coordinates when a valid GPS fix is available.
5. **Buzzer Alert Mechanism:** Implemented a temperature-based alert using a configured threshold of 34°C.
6. **Web Dashboard:** Developed a browser-based interface to receive and display sensor readings through serial communication.
7. **Historical Data and Export:** Included support for reviewing stored readings and exporting available data as CSV files where implemented.
8. **Future Expansion:** Established a foundation for integrating particulate matter monitoring and additional environmental analysis.

These achievements demonstrate the feasibility of combining embedded hardware and a web interface to create a practical environmental monitoring system.

6.2 FUTURE WORK

Although EcoGuard provides the core functions for environmental monitoring, several improvements can enhance its accuracy, usability, and functionality.

6.2.1 PMS5003 Particulate Matter Integration

The PMS5003 can be integrated to measure $PM_{2.5}$ and PM_{10} concentrations. Future work should include correct wiring, UART communication, data-frame parsing, and validation of the reported measurements before displaying them on the dashboard.

6.2.2 Improved Sensor Accuracy

The MQ-135 can be calibrated and evaluated under suitable conditions to improve the interpretation of its readings. Future development could include appropriate calibration procedures and additional sensors to obtain more reliable environmental measurements.

6.2.3 Cloud Integration and Remote Monitoring

The system could be connected to a cloud platform to allow users to access environmental readings remotely. Cloud storage could also support longer-term data retention, centralized monitoring, and access to historical measurements from different locations.

6.2.4 AI-Based Environmental Analysis

After collecting sufficient validated sensor data, machine learning techniques could be explored to identify trends and estimate future environmental conditions. An Environmental Health Risk Score (EHRS) or personalized recommendations should be developed only with a clearly defined method and appropriate validation.

6.2.5 Mobile Notifications and Location Mapping

Future versions could provide mobile notifications for configured alerts and display valid GPS coordinates on an interactive map. These features could help users identify where readings were collected and respond to relevant changes.

6.2.6 Enhanced Dashboard and Reporting

The dashboard could be improved with interactive graphs, date-based filtering, additional report formats, and comparisons between historical readings. These features would make it easier to identify trends and examine changes over time.

6.2.7 Power Efficiency and Portable Deployment

Power consumption could be reduced through optimized sensor sampling, efficient communication, and suitable power-management techniques. This would help support portable or battery-powered environmental monitoring applications.

SUMMARY

EcoGuard demonstrates the integration of an ESP32, environmental sensors, GPS, a buzzer, and a web dashboard into a single monitoring system. It supports the collection and display of available sensor readings and provides a local warning when the configured high-temperature threshold is exceeded.

The project provides a foundation for future improvements, including PMS5003 integration, improved sensor calibration, cloud-based monitoring, and AI-based environmental analysis. Further testing and validation will be important to ensure reliable measurements and meaningful results.

Overall, EcoGuard offers a practical starting point for an expandable IoT environmental monitoring solution that can be improved to support more comprehensive environmental observation in the future.

CHAPTER 7

SOURCE CODE

ESP32 Main Code

```
#include <DHT.h>
#include <TinyGPSPlus.h>
#include <HardwareSerial.h>
#define DHT_PIN 4
#define DHT_TYPE DHT22
#define MQ135_PIN 34
#define BUZZER_PIN 25
DHT dht(DHT_PIN, DHT_TYPE);
TinyGPSPlus gps;
HardwareSerial gpsSerial(2);
const float TEMP_THRESHOLD = 34.0;
bool buzzerOn = false;
void setup() {
  Serial.begin(115200);
  gpsSerial.begin(9600, SERIAL_8N1, 26, 27);
  dht.begin();
  pinMode(MQ135_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  Serial.println("EcoGuard Environmental Monitoring Started");
}
void loop() {
```

```
while (gpsSerial.available()) {
  gps.encode(gpsSerial.read());
}
float temperature = dht.readTemperature();
float humidity = dht.readHumidity();
int gasValue = analogRead(MQ135_PIN);
if (!isnan(temperature) && !isnan(humidity)) {
  Serial.print("TEMPERATURE: ");
  Serial.println(temperature);
  Serial.print("HUMIDITY: ");
  Serial.println(humidity);
  Serial.print("MQ135: ");
  Serial.println(gasValue);
  if (temperature > TEMP_THRESHOLD) {
    digitalWrite(BUZZER_PIN, HIGH);
    if (!buzzerOn) {
      Serial.println("BUZZER: ON - HIGH TEMPERATURE");
      buzzerOn = true;
    }
  } else {
    digitalWrite(BUZZER_PIN, LOW);
    if (buzzerOn) {
      Serial.println("BUZZER: OFF");
      buzzerOn = false;
    }
```

```
    }
  } else {
    Serial.println("DHT22: SENSOR READ ERROR");
  }
  if (gps.location.isValid()) {
    Serial.print("LATITUDE: ");
    Serial.println(gps.location.lat(), 6);
    Serial.print("LONGITUDE: ");
    Serial.println(gps.location.lng(), 6);
  } else {
    Serial.println("GPS: NO FIX");
  }
  Serial.println("--------------------");
  delay(2000);
}
```

Html.css

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>EcoGuard | Environmental Monitor</title>
  <style>
    :root {
      --venice: #16587B; --rock: #84B3CE; --merino: #F5EEDD;
```

```
  --ink: #163247; --muted: #607482; --surface: #fffdf8;
  --line: #d8e2e7; --danger: #b93838; --good: #24734b;
}
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, Helvetica, sans-serif; background: var(--merino); color: var(--ink); }
header { background: var(--venice); color: white; padding: 22px clamp(16px, 4vw, 48px); }
header .top { display:flex; justify-content:space-between; align-items:center; gap:16px; flex-wrap:wrap; }
h1 { margin:0; font-size: clamp(24px, 3vw, 34px); letter-spacing:.2px; }
header p { margin:7px 0 0; color:#e1edf3; }
.status { border:1px solid #b7d5e4; padding:8px 12px; border-radius:99px; font-size:13px; }
main { max-width:1200px; margin:auto; padding:24px clamp(14px, 3vw, 32px) 44px; }
.toolbar { display:flex; gap:10px; flex-wrap:wrap; margin-bottom:20px; }
button { cursor:pointer; border:0; border-radius:10px; padding:11px 15px; background:var(--venice); color:white; font-weight:700; }
button.secondary { background:#dceaf0; color:var(--venice); }
button.warn { background:#8f3131; }


.
<article class="card">
  <h2>Latest update</h2>
  <div class="value" id="lastUpdate" style="font-size:22px">Waiting for data</div>
  <div class="sub" id="dataSource">Source: none</div>
  <p class="foot">Data is saved in this browser's local storage. It is not automatically uploaded to a cloud server.</p>
</article>
```

```
  </section>


  <section class="section card">
    <h2>Sensor history and serial output</h2>
    <div class="tabs">
      <button class="active" data-tab="historyPanel">History</button>
      <button data-tab="rawPanel" class="secondary">Raw serial</button>
      <button data-tab="aboutPanel" class="secondary">Notes</button>
    </div>
    <div id="historyPanel" class="panel active">
                        <div class="table-wrap"><table><thead><tr><th>Time</th><th>Temp (°C)</th><th>Humidity (%)</th><th>MQ-135 raw</th><th>GPS</th></tr></thead><tbody id="historyRows"><tr><td colspan="5">No readings yet.</td></tr></tbody></table></div>
    </div>
    <div id="rawPanel" class="panel"><div id="rawOutput" class="raw">Serial output will appear here.</div></div>
    <div id="aboutPanel" class="panel">
      <p><b>Current implementation:</b> ESP32 + DHT22 + MQ-135 + NEO-6M GPS + buzzer, with browser-based serial display.</p>
       <p><b>Not included as verified functionality:</b> PMS5003 particulate measurement, calibrated AQI/ppm, cloud storage, and AI prediction. Add these only after implementing and validating the hardware/software.</p>
      <p><b>GPS:</b> The NEO-6M often needs a clear view of the sky to obtain a fix.</p>
    </div>
  </section>
  <p class="foot">EcoGuard prototype dashboard. Readings depend on sensor wiring, calibration, power supply, and environmental conditions.</p>
</main>
```

```
<script>
(() => {
  const $ = id => document.getElementById(id);
  const STORAGE_KEY = "ecoguard_history_v1";
  let history = loadHistory();
  let port = null, reader = null, keepReading = false, demoTimer = null;
  let latest = { temperature: null, humidity: null, mq135: null, latitude: null, longitude: null };
  let buzzerActive = false;
  let lineBuffer = "";

  function loadHistory() {
    try {
      const data = JSON.parse(localStorage.getItem(STORAGE_KEY) || "[]");
      return Array.isArray(data) ? data.slice(-500) : [];
    } catch { return []; }
  }
  function persist() {
    try { localStorage.setItem(STORAGE_KEY, JSON.stringify(history.slice(-500))); } catch(e) {
logRaw("Storage error: " + e); }
  }
  function fmt(v, digits=1) { return Number.isFinite(v) ? v.toFixed(digits) : "--"; }
  function setStatus(text) { $("connectionStatus").textContent = "● " + text; }
  function updateDisplay(source) {
    $("temperature").innerHTML = `${fmt(latest.temperature)} <span class="unit">°C</span>`;
```

```
  $("humidity").innerHTML = `${fmt(latest.humidity)} <span class="unit">%</span>`;
  $("mq135").textContent = Number.isFinite(latest.mq135) ? Math.round(latest.mq135) : "--";
  if (Number.isFinite(latest.latitude) && Number.isFinite(latest.longitude)) {
    $("gpsStatus").textContent = "Fix acquired";
    $("coordinates").textContent = `${latest.latitude.toFixed(6)}, ${latest.longitude.toFixed(6)}`;
  } else {
    $("gpsStatus").textContent = "No fix";
    $("coordinates").textContent = "Coordinates unavailable";
  }
  $("lastUpdate").textContent = new Date().toLocaleTimeString();
  $("dataSource").textContent = "Source: " + source;
}
function logRaw(line) {
  const el = $("rawOutput");
  if (el.textContent === "Serial output will appear here.") el.textContent = "";
  el.textContent += line + "\n";
  const lines = el.textContent.split("\n");
  if (lines.length > 150) el.textContent = lines.slice(-150).join("\n");
  el.scrollTop = el.scrollHeight;
}
function updateAlert(active, message) {
  buzzerActive = active;
  const box = $("alertBox");
  box.classList.toggle("active", active);
  box.textContent = message;
```

```
  if (active && "Notification" in window && Notification.permission === "granted") {
    try { new Notification("EcoGuard: High temperature", { body: message }); } catch {}


function saveReading(source) {
  if (![latest.temperature, latest.humidity, latest.mq135].some(Number.isFinite)) return;
  const row = { time: new Date().toISOString(), temperature: latest.temperature,
    humidity: latest.humidity, mq135: latest.mq135,
    latitude: latest.latitude, longitude: latest.longitude, source };
  history.push(row); history = history.slice(-500); persist(); renderHistory();
  updateDisplay(source);
}
function renderHistory() {
  const tbody = $("historyRows");
  if (!history.length) { tbody.innerHTML = '<tr><td colspan="5">No readings yet.</td></tr>'; return; }
  tbody.innerHTML = history.slice().reverse().slice(0,100).map(r => {
    const gps = Number.isFinite(r.latitude) && Number.isFinite(r.longitude) ?
      `${r.latitude.toFixed(5)}, ${r.longitude.toFixed(5)}` : "No fix";
    return `<tr><td>${new Date(r.time).toLocaleString()}</td><td>${fmt(r.temperature)}</td><td>${fmt(r.humidity)}</td><td>${Number.isFinite(r.mq135) ? Math.round(r.mq135) : "--"}</td><td>${gps}</td></tr>`;
  }).join("");
  let m;
    if ((m = s.match(/^TEMPERATURE:\s*(-?\d+(?:\.\d+)?)/i))) latest.temperature = Number(m[1]);
  else if ((m = s.match(/^HUMIDITY:\s*(-?\d+(?:\.\d+)?)/i))) latest.humidity = Number(m[1]);
  else if ((m = s.match(/^MQ135:\s*(\d+)/i))) latest.mq135 = Number(m[1]);
```

```
    else if ((m = s.match(/^LATITUDE:\s*(-?\d+(?:\.\d+)?)/i))) latest.latitude = Number(m[1]);
    else if ((m = s.match(/^LONGITUDE:\s*(-?\d+(?:\.\d+)?)/i))) latest.longitude = Number(m[1]);
    else if (/^GPS:\s*NO FIX/i.test(s)) { latest.latitude = null; latest.longitude = null; }
    else if (/^BUZZER:\s*ON\s*-\s*HIGH TEMPERATURE/i.test(s)) {
      updateAlert(true, "HIGH TEMPERATURE: ESP32 reports that the physical buzzer is ON.");
    } else if (/^BUZZER:\s*OFF/i.test(s)) {
      updateAlert(false, "No active high-temperature buzzer alert reported by the ESP32.");
    }
    if (/^-{5,}$/.test(s)) {
      updateDisplay("ESP32 serial");
      saveReading("ESP32 serial");
renderHistory();
})();
</script>
</body>
</html>
```

REFERENCE:

[1] S. Singh, M. Kumar, V. Sengar, A. Kumar, K. Abhishek, R. Kaushik, E. Kelkar, A. Gupta, and A. Bhargava, "Development of an Air Pollution Monitoring System," *Procedia Computer Science*, vol. 260, pp. 645–654, 2025, doi: 10.1016/j.procs.2025.03.243.

[2] J. Rosa-Bilbao, F. S. Butt, D. Merkl, M. F. Wagner, J. Schäfer, and J. Boubeta-Puig, "IoT-Based Indoor Air Quality Management System for Intelligent Education Environments," *IEEE Internet of Things Journal*, vol. 12, no. 11, pp. 18031–18041, 2025.

[3] C. Ou, F. Li, J. Zhang, P. Jiang, W. Li, S. Kong, J. Guo, W. Fan, and J. Zhao, "Multi-scenario PM2.5 distribution and dynamic exposure assessment of university community residents: Development and application of intelligent health risk management system integrated low-cost sensors," *Environment International*, vol. 185, Art. no. 108539, 2024, doi: 10.1016/j.envint.2024.108539.

[4] M. M. Abdelmalek, H. Mahmoud, and H. Shokry, "Prognosis of air quality index and air pollution using machine learning techniques," *Scientific Reports*, vol. 15, Art. no. 25890, 2025,

[5] A. Bekkar, B. Hssina, N. Abekiri, S. Douzi, and K. Douzi, "Real-time AIoT platform for monitoring and prediction of air quality in Southwestern Morocco," *PLOS ONE*, vol. 19, no. 8, Art. no. e0307214, 2024, doi: 10.1371/journal.pone.0307214.

[6] C. Mountzouris, G. Protopsaltis, and J. Gialelis, "Toward Personalized Short-Term PM2.5 Forecasting Integrating a Low-Cost Wearable Device and an Attention-Based LSTM," *Air*, vol. 3, no. 4, Art. no. 29, 2025, doi: 10.3390/air3040029.

[7] O. Yildiz and H. S. Sucuoglu, "Development of Real-Time IoT-Based Air Quality Forecasting System Using Machine Learning Approach," *Sustainability*, vol. 17, Art. no. 8531, 2025, doi: 10.3390/su17198531.

[8] M. Rajesh, R. Ganesh Babu, U. Moorthy, and S. V. Easwaramoorthy, "Machine learning-driven framework for realtime air quality assessment and predictive environmental health risk mapping," *Scientific Reports*, vol. 15, Art. no. 28801, 2025.

[9] G. Navaneeth, M. Praneeth, M. Khasim, and Y. M. Roopa, "A Personalized Healthcare Model for Air Pollution Monitoring and Prediction Using Machine Learning," in *Proceedings of the 1st International Conference on Research and Development in Information, Communication, and Computing Technologies (ICRDICCT)*, 2025, pp. 575–579.

[10] R. Manikandan, G. Gobinath, G. Gokul, and D. Boobalan, “AI-Driven IoT Framework for Real-Time Air Quality Monitoring and Stress Correlation Analysis,” in *Proceedings of the 1st International Conference on Research and Development in Information, Communication, and Computing Technologies* (ICRDICCT’25), vol. 3, pp. 87–92, 2025